PRODUCT IMPLEMENTATION PROPOSAL

Accounting in Judibas

From ERPNext workflows to a Judibas product

A step-by-step implementation proposal for Laravel + Vue 3, with one master tree and twelve individual step diagrams.

12 STEPS / 13 DIAGRAMS / DELIVERY CHECKPOINTS

Prepared for PM, engineering and finance review

21 October 2026
Planning document; implementation has not started

> Primary route: build a Judibas-native Accounting product using independently specified ERPNext-style workflows. The API integration route is explained separately. ERPNext Python modules cannot simply be pasted into the Laravel application.

First decide what “copy” means

Judibas currently uses Laravel 12, PHP, Vue 3 and a shared PostgreSQL database. ERPNext depends on the Frappe application framework. Similar-looking screens do not make the two application runtimes interchangeable. [1]

Route	How it works	Main trade-off
A. Native Judibas product	Implement accounting rules and UI in Laravel/Vue; keep the existing shared database.	One user experience, but Judibas must build and validate the accounting engine.
B. ERPNext integration	Run ERPNext separately; Judibas calls authorized APIs and maps business identities.	Reuses the engine, but adds separate operations, mapping and synchronization.

Recommendation for your stated requirement

Route A matches your existing product pattern and shared database preference. Select it only with an accountant involved and phased acceptance. If reducing custom ledger engineering is the priority, first prove Route B with a small integration pilot. Neither route is an automatic module copy.

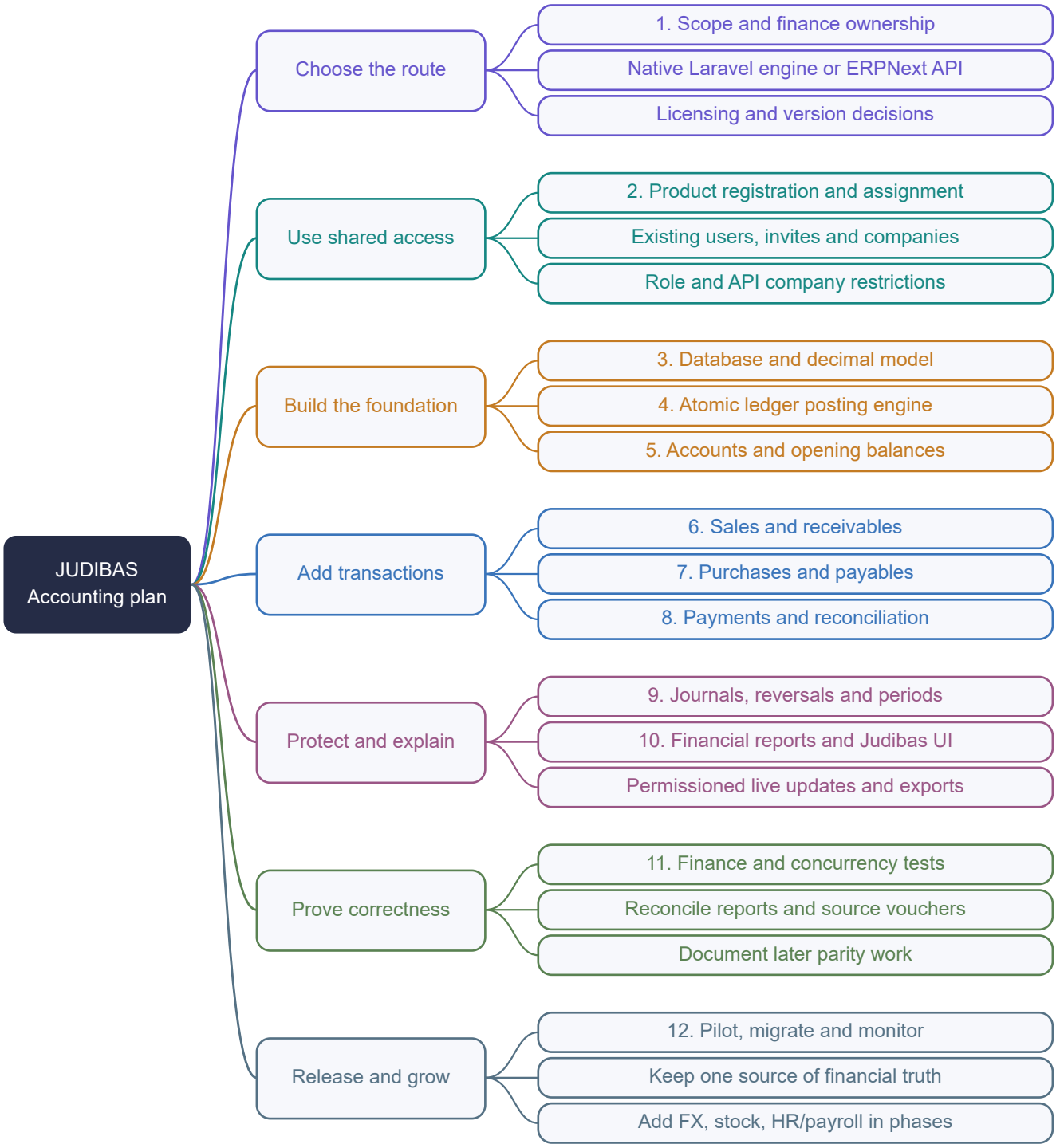
> ERPNext publishes a GPLv3 license. Before reusing its actual code, templates or assets, review the applicable license obligations for the intended distribution and deployment. API separation is an architectural choice, not a blanket licensing exemption. [3]

All steps and database suggestions below are proposed Judibas implementation decisions, not claims that they already exist.

Sources: [1] Frappe architecture, [2] Frappe REST API, [3] ERPNext license

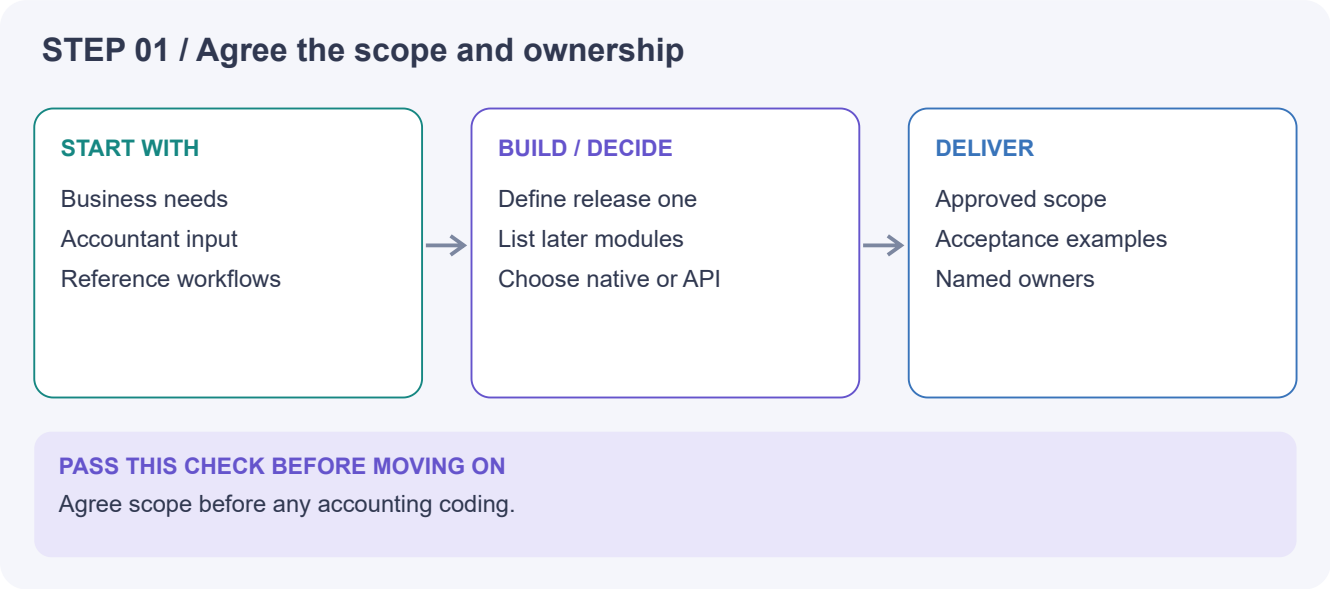
Overall implementation tree

Follow the numbered sequence. Ledger correctness is established before transaction screens and dashboards.



PROPOSED IMPLEMENTATION / STEP 1 OF 12

Step 01 / Agree the scope and ownership



What we do

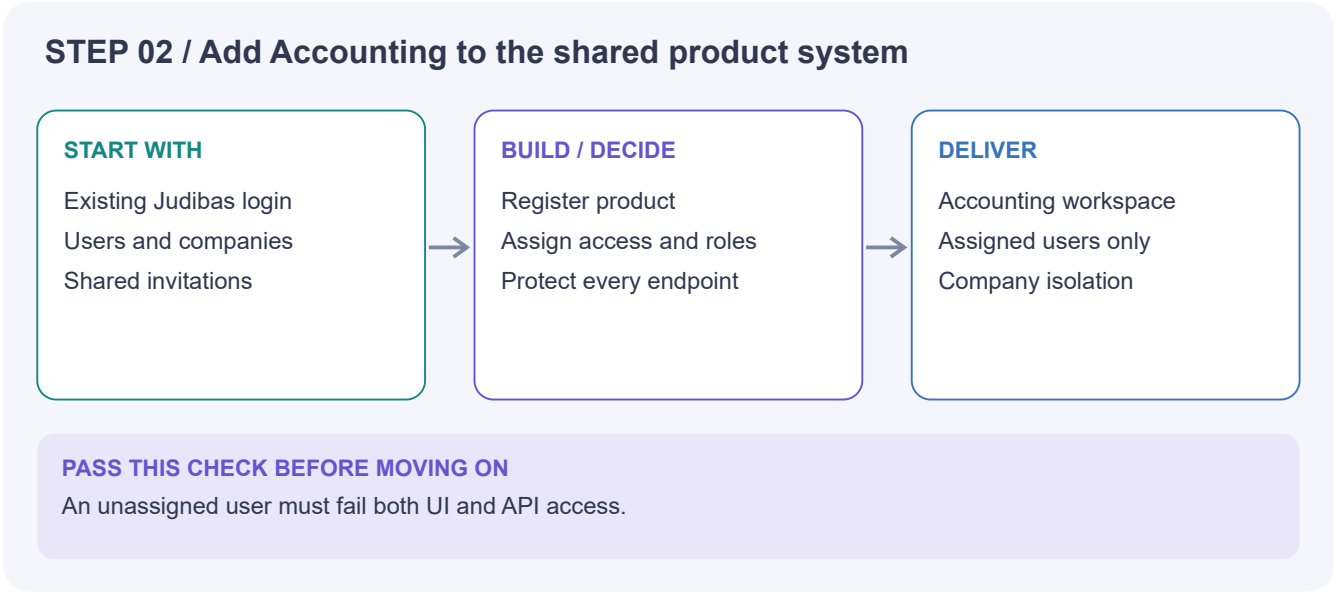
- Inventory the required screens, reports, document fields and business rules. Use ERPNext as the functional reference, with a pinned release for comparison.
- Name a finance owner to approve account mappings, opening balances, taxes and worked examples. PM owns scope; engineering owns implementation.
- Define release one as single-currency service accounting: accounts, invoices, bills, payments, journals and essential reports. Do not label this complete ERPNext parity.

> **Completion checkpoint**
> Output: a signed scope and feature matrix with Required / Later / Not applicable decisions.

Example: decide whether cheque tracking is only a reference field or a full issued-to-cleared lifecycle.

PROPOSED IMPLEMENTATION / STEP 2 OF 12

Step 02 / Add Accounting to the shared product system



What we do

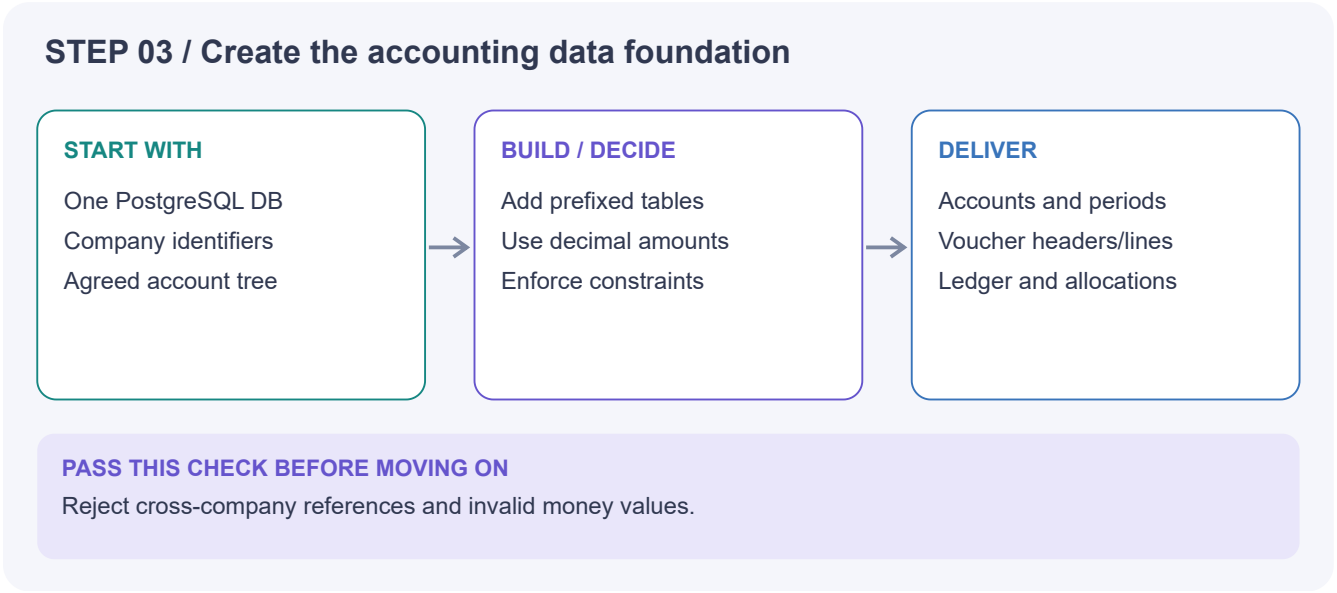
- Register an Accounting product using the same catalogue, application switcher, shared login and access-assignment pattern as other Judibas products.
- Keep account creation, invitations and company creation in Administration. Show accounting configuration and transactions inside the Accounting workspace.
- Define separate abilities for view, create, submit, pay, journal, cancel, export and period management. Enforce company access on every server action, not only in Vue.

> **Completion checkpoint**
> Output: a working empty product shell with permission checks and role tests.

Example: a cashier can record authorized payments without acquiring journal-adjustment or cancellation powers.

PROPOSED IMPLEMENTATION / STEP 3 OF 12

Step 03 / Create the accounting data foundation



What we do

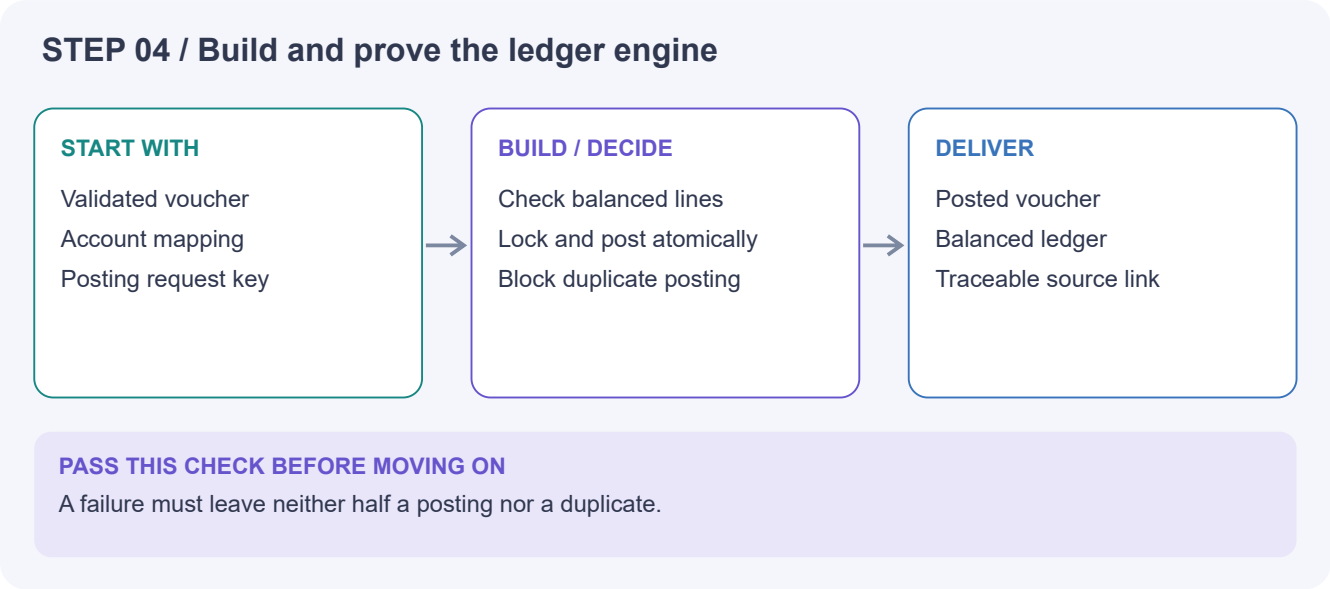
- Keep the existing shared PostgreSQL database. Add accounting-specific tables and indexes rather than changing unrelated product records.
- Model periods, accounts, parties, vouchers, journal lines, invoice items, payment allocations, audit and posting requests. Proposed table names are design choices, not ERPNext table copies.
- Use fixed-precision decimal arithmetic with an agreed rounding policy. Enforce company-consistent references, unique document numbers and idempotency keys; store currency codes explicitly.

> **Completion checkpoint**
> Output: reviewed migrations, relationships, constraints and a data dictionary.

Example: one voucher header belongs to company A; every posting account and invoice allocation must also belong to company A.

PROPOSED IMPLEMENTATION / STEP 4 OF 12

Step 04 / Build and prove the ledger engine



What we do

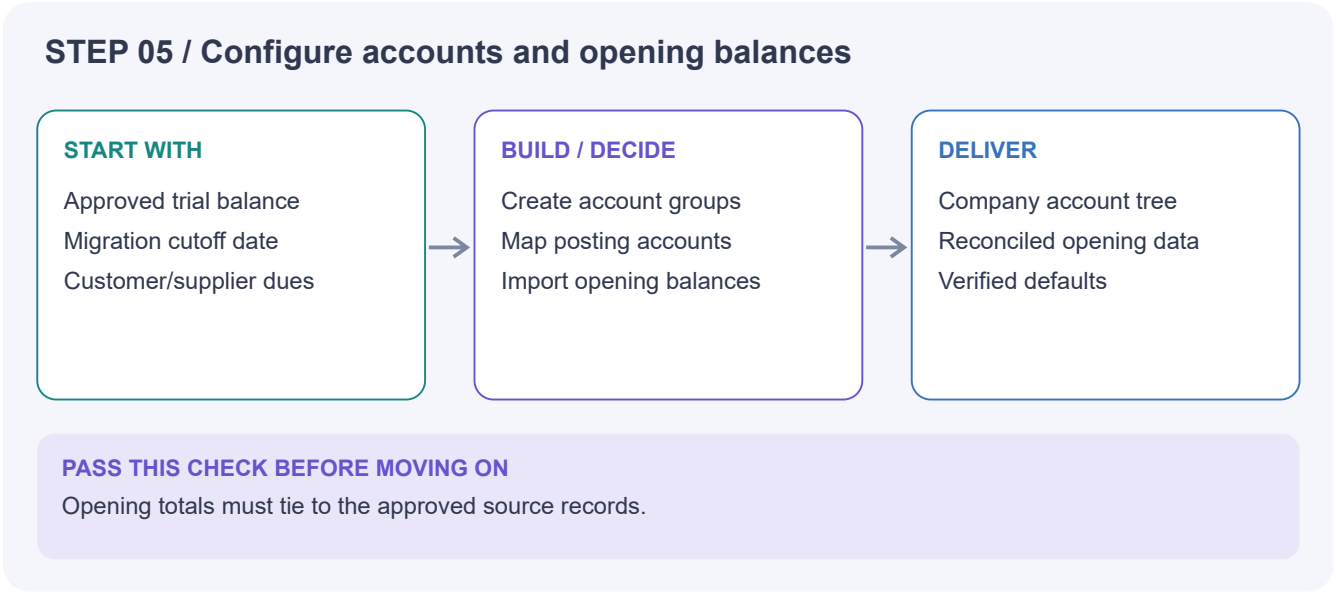
- Create one server-side posting service shared by invoices, payments and journals. Check permissions, company, accounts, period, currency and balanced totals before posting.
- Lock the relevant records and commit the voucher state, ledger lines and audit trail within one database transaction. Retried requests must return or identify the prior result.
- Keep financial posting synchronous and atomic. Queue notifications and broadcasts only after commit; a failed broadcast must not make a successful financial posting look failed.

> **Completion checkpoint**
> Output: a tested posting engine before building the transaction screens.

Example: two simultaneous Submit clicks must result in one posted invoice and one set of ledger lines.

PROPOSED IMPLEMENTATION / STEP 5 OF 12

Step 05 / Configure accounts and opening balances



What we do

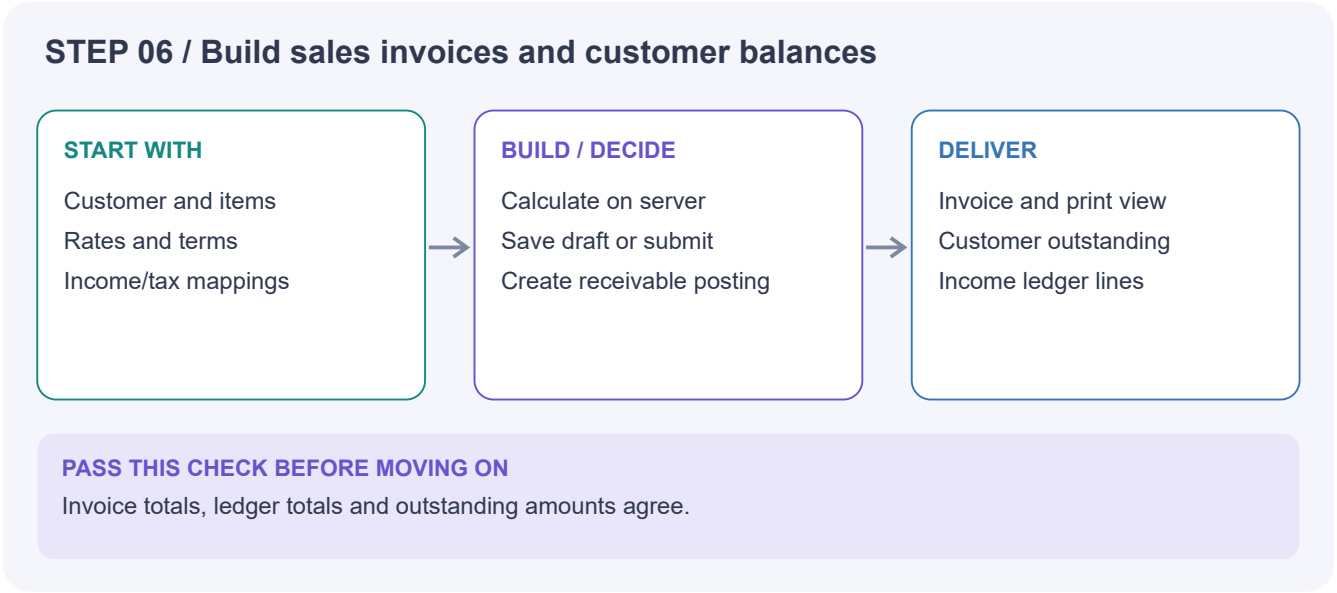
- Build account-tree management with group versus posting accounts. Add company defaults, fiscal periods, currency and controlled document-number sequences.
- Prepare customer/supplier records and bank/cash accounts. Reuse shared contacts only when their identity and company meaning match; do not silently merge unrelated parties.
- Load opening ledger balances and invoice-level outstanding amounts through a deliberate migration process. Reconcile them together so opening receivables/payables are not counted twice.

> **Completion checkpoint**
> Output: a finance-approved opening balance pack and master-data checklist.

Example: the customer-control account total must agree with the imported customer outstanding detail.

PROPOSED IMPLEMENTATION / STEP 6 OF 12

Step 06 / Build sales invoices and customer balances



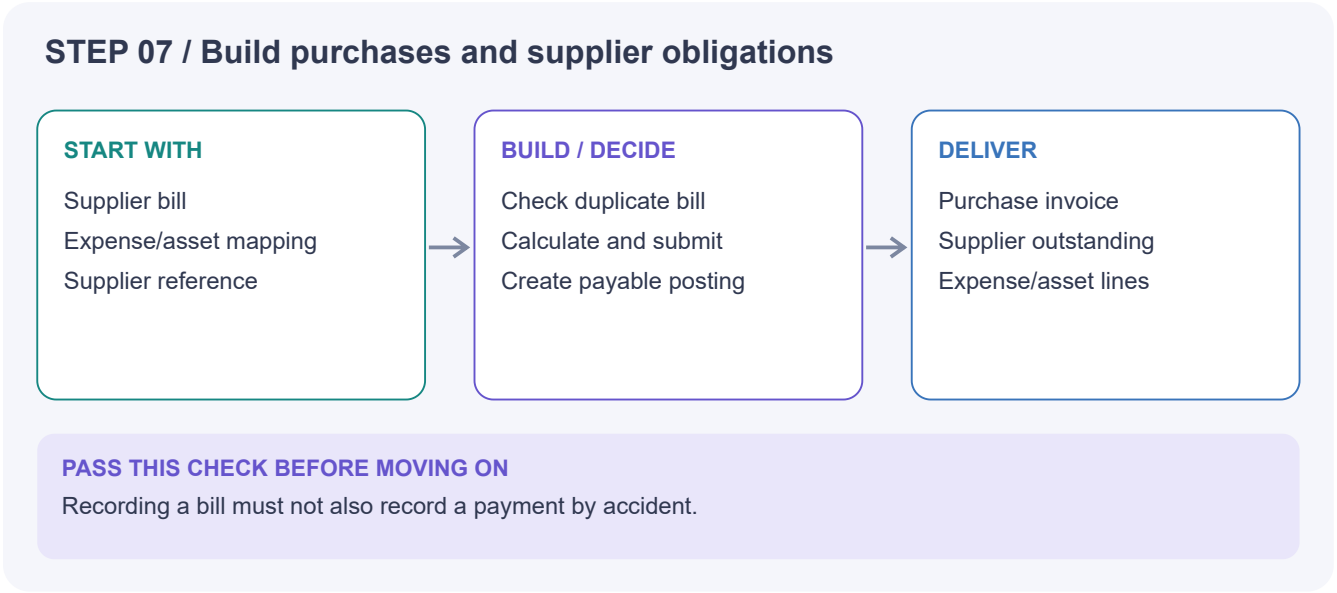
What we do

- Implement a Judibas-style invoice list, form, preview and print/PDF view. Clearly distinguish a draft, a posted invoice and its payment state.
- Recalculate quantities, rates, discounts, taxes and rounding on the server. Snapshot the posted document details required for historical reporting.
- Post receivables, income and applicable tax lines through the ledger service. Keep payment allocation separate; include controlled credit/return handling in the agreed release scope.

> **Completion checkpoint**
> Output: draft-to-posted invoicing with correct customer balances and printable documents.

Example: a Rs 100,000 service invoice creates Rs 100,000 receivable; cash remains unchanged until payment.

Step 07 / Build purchases and supplier obligations



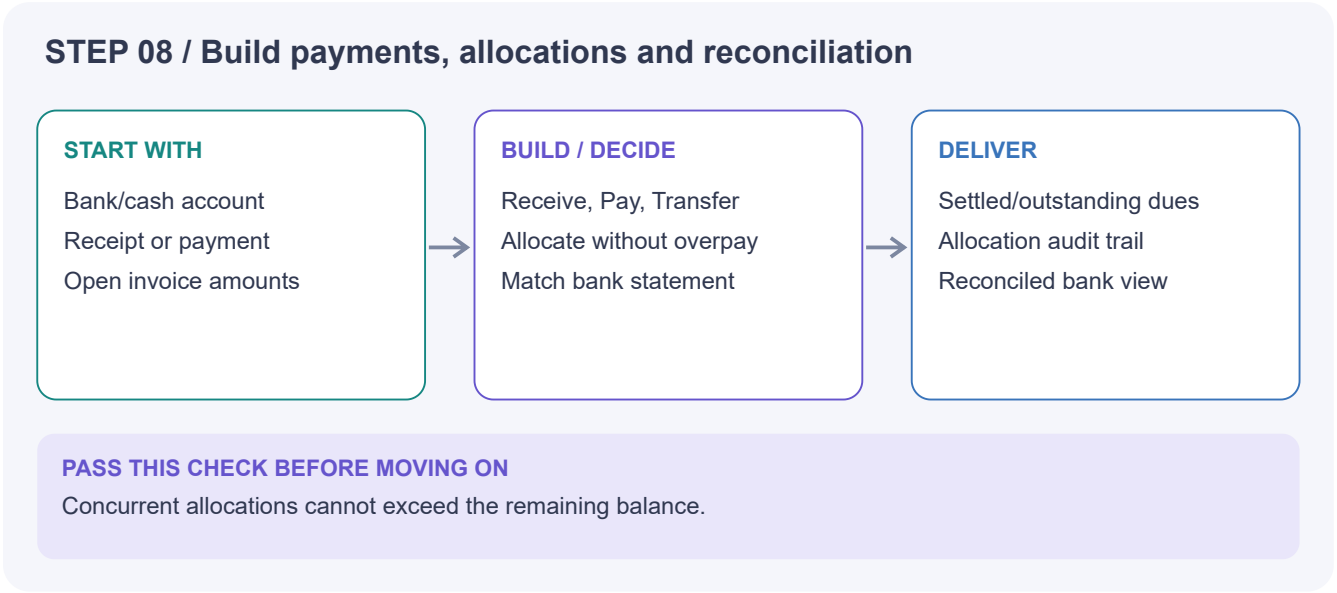
What we do

- Implement supplier bills using the same draft, review, submit and historical-preview pattern. Retain supplier references, dates, attachments and source evidence.
- Post expense or asset and payable lines through the same ledger engine. Add duplicate supplier-reference checks and clear due-date information.
- Begin with service bills. Stock receipts, stock valuation and inventory cost postings require a separately tested extension; do not imitate them with generic expense entries.

> **Completion checkpoint**
> Output: supplier bills and accurate payable detail with supporting evidence.

Example: a Rs 30,000 unpaid service bill increases expense and payables; the bank balance does not move.

Step 08 / Build payments, allocations and reconciliation



What we do

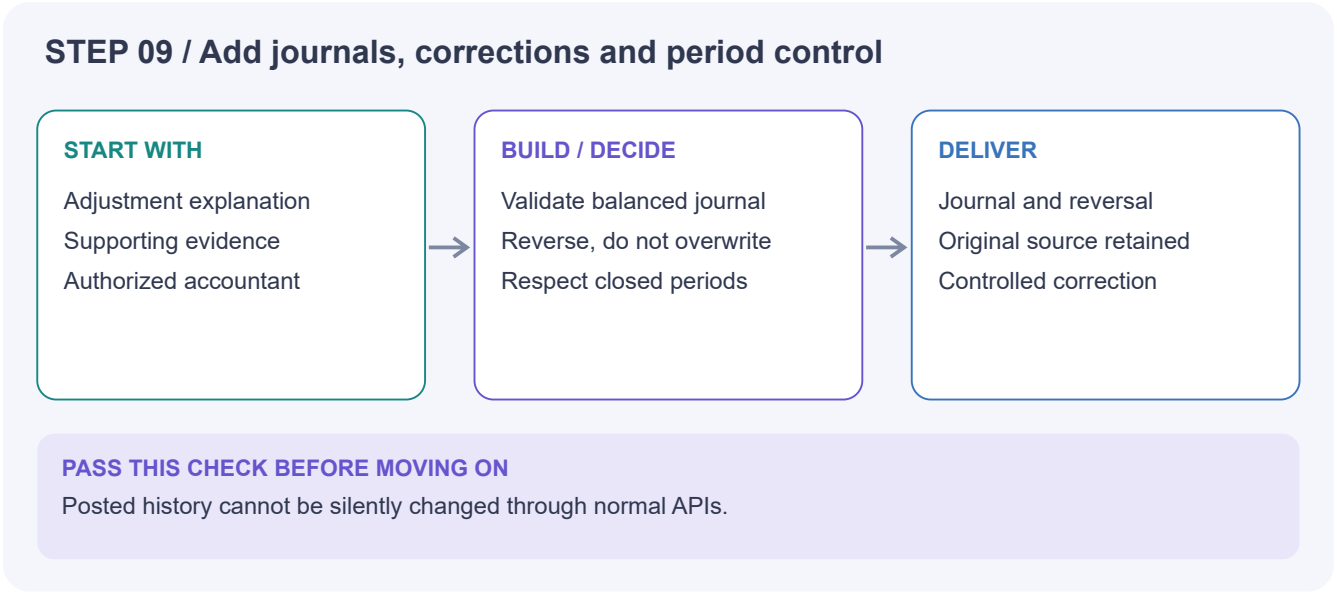
- Implement customer receipts, supplier payments and internal transfers. Add advances and unallocated amounts only with explicit later-allocation rules.
- Lock invoices and allocation records during settlement. Define policies for refunds, small differences, fees, overpayments and reversals.
- Provide CSV/manual bank-statement import and reconciliation first. Add provider connectivity only after availability is verified. Keep invoice allocation and bank reconciliation as separate operations.

> **Completion checkpoint**
> Output: full/partial settlement, internal transfers and a bank reconciliation workflow.

Example: receive Rs 60,000 against a Rs 100,000 invoice; outstanding becomes Rs 40,000 without adding new revenue.

PROPOSED IMPLEMENTATION / STEP 9 OF 12

Step 09 / Add journals, corrections and period control



What we do

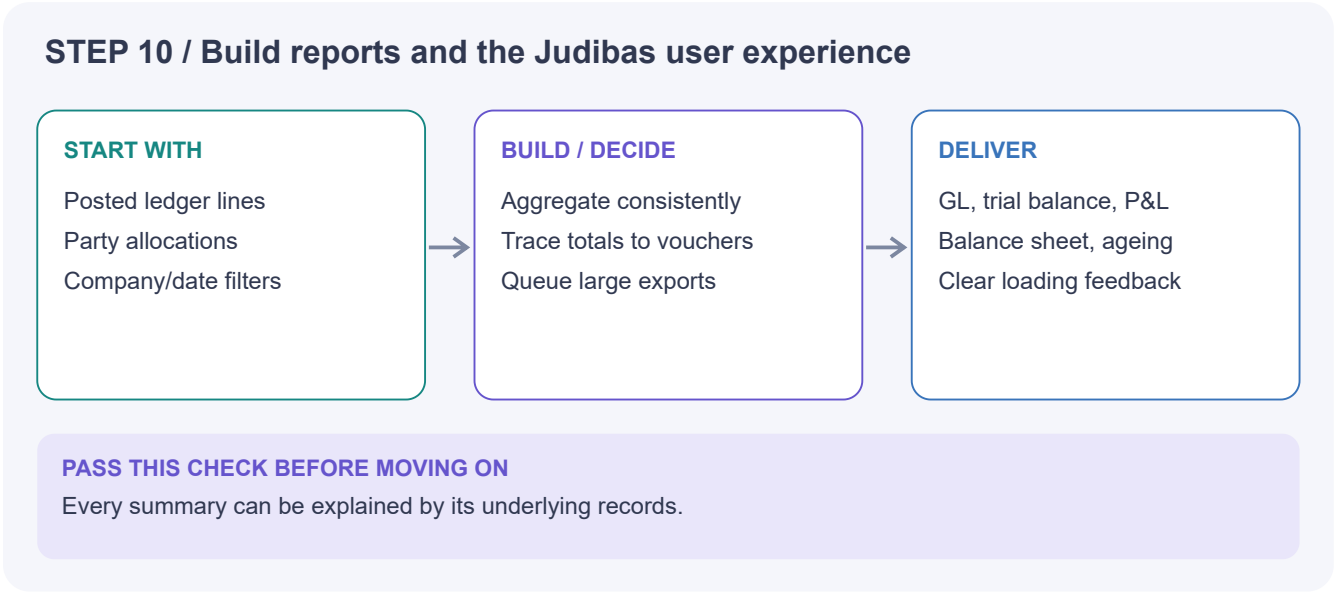
- Implement balanced manual journals for approved adjustments with evidence and clear account/party selections. Require specific journal permissions.
- Implement reversal-linked correction rather than direct editing of posted amounts. Decide how related allocations, credits and dependent documents are reversed or blocked.
- Close periods with authorized reopening controls. Audit submit, cancellation, correction and export events with actor, company, time and meaningful business descriptions.

> **Completion checkpoint**
> Output: traceable correction paths, period controls and understandable audit history.

Example: cancelling an incorrect invoice preserves the original posting and records an authorized opposite effect.

PROPOSED IMPLEMENTATION / STEP 10 OF 12

Step 10 / Build reports and the Judibas user experience



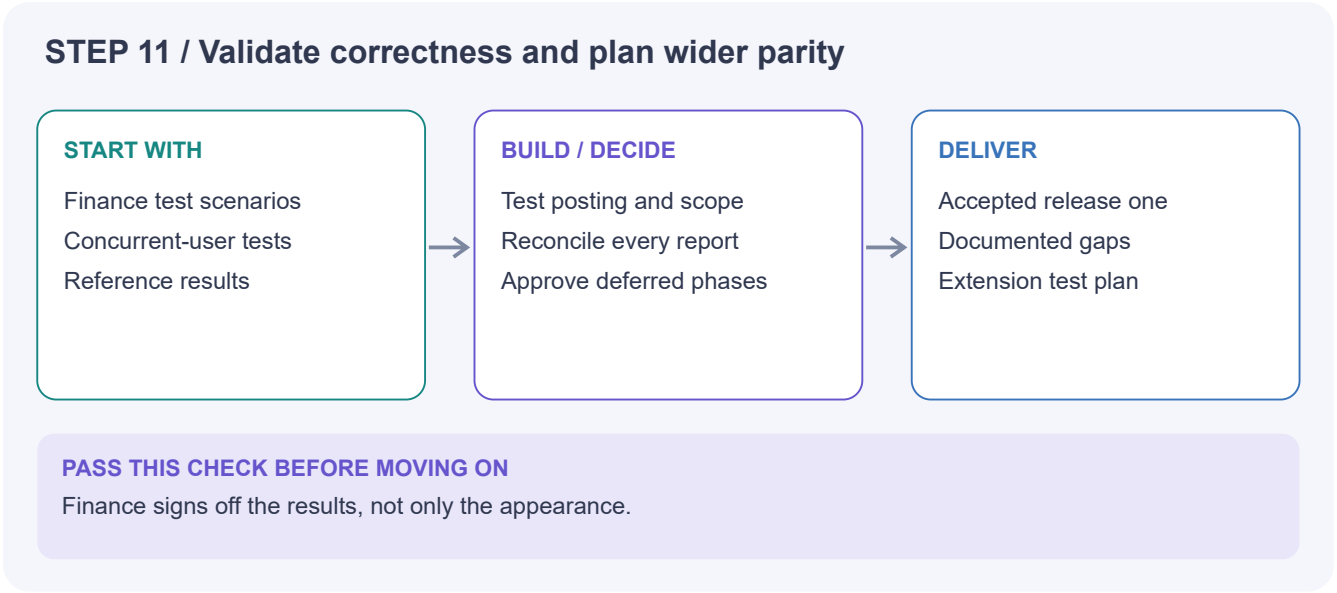
What we do

- Build General Ledger, Trial Balance, Balance Sheet, Profit and Loss, receivables/payables ageing and account/party statements. Validate cash-flow classification before releasing that report.
- Keep totals based on the ledger and outstanding figures based on valid allocations. Apply the same company, date, permission and status filters to screens and exports.
- Match Judibas navigation, themes, tables and dialogs. Use role-relevant dashboards, clickable totals, in-button loaders, pagination and readable previews. Broadcast permitted changes after commit using Reverb/Pusher.

> **Completion checkpoint**
> Output: reconciled financial reports and a responsive Accounting workspace.

Example: clicking an outstanding customer total opens the exact unpaid invoice detail visible to that user.

Step 11 / Validate correctness and plan wider parity



What we do

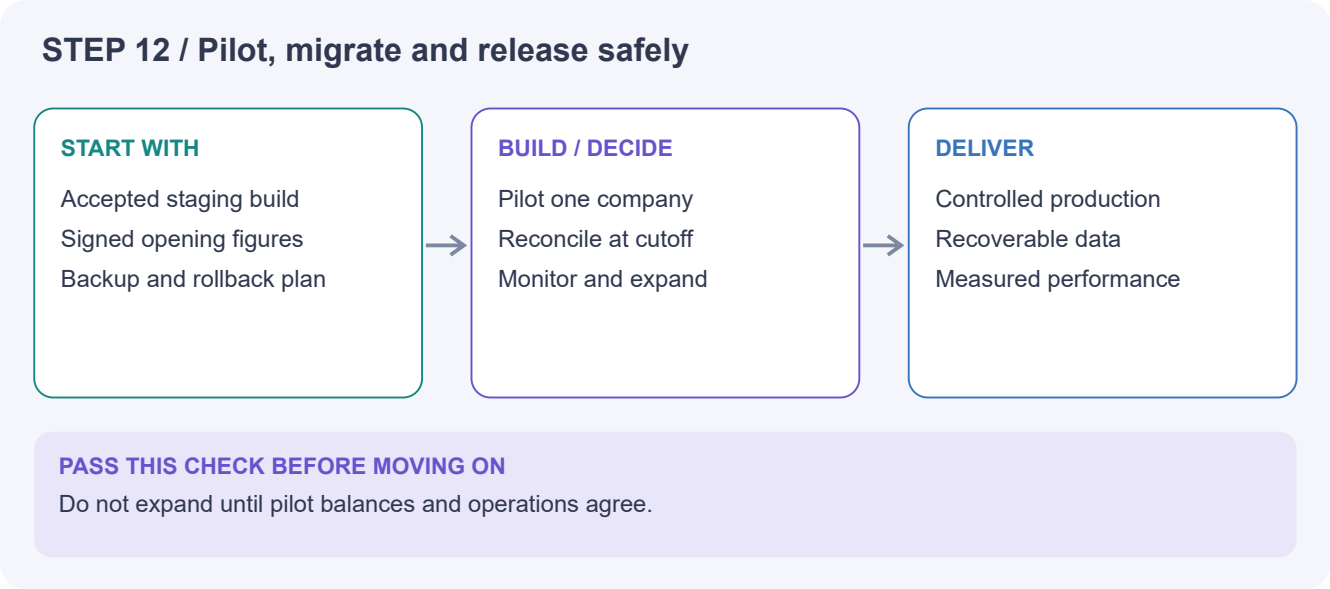
- Test rounding, partial payments, advances, retries, concurrent posting, credit notes, cancellation, period locks and unauthorized cross-company access. Use meaningful database and browser tests.
- Compare representative examples against independently calculated expected results and a configured reference installation. Test backups, exports, worker failures and real-time access.
- Track wider parity explicitly: multi-currency/FX, dimensions, budgets, finance books, stock/COGS, assets, payroll, tax localization and cheque lifecycle each need dedicated specifications and validation.

> **Completion checkpoint**
> Output: an acceptance pack, known limitations and an approved next-phase backlog.

Example: a failed notification must not duplicate a payment or persuade the operator to post it again.

PROPOSED IMPLEMENTATION / STEP 12 OF 12

Step 12 / Pilot, migrate and release safely



What we do

- Pilot with one company and a small finance team. Import a controlled opening dataset and run agreed scenarios before wider employee access.
- Choose the financial source of truth at cutover. Avoid manual dual entry into two live ledgers without a controlled reconciliation process. Restore-test backups before launch.
- Deploy supervised workers, scheduler and Reverb; monitor queries, locks, failed jobs, disk and response times. Load-test the agreed concurrency assumption rather than guaranteeing capacity from server specifications.

> **Completion checkpoint**
> Output: a reconciled pilot, go-live checklist, recovery runbook and support ownership.

Example: plan for 400 registered users, but measure actual simultaneous active users and expensive report workloads.

If we choose ERPNext integration

This is a separate route from the twelve-step native plan. Frappe exposes REST APIs; the chosen ERPNext version and permitted document lifecycle must be tested before relying on them. [2]

Sequence	Integration work required
1. Deploy a reference service	Operate ERPNext with its own supported application runtime and database. Do not let Laravel write its internal tables directly.
2. Map identities and companies	Map Judibas users, company IDs, customers, suppliers and accounts to the ERPNext records. Shared sign-in requires deliberate identity integration.
3. Choose the financial source of truth	ERPNext owns authoritative ledger records. Judibas stores integration mappings and any explicitly identified cache.
4. Build a server-side adapter	Keep credentials on the server. Validate the actor and company; do not expose a powerful shared API token to the browser.
5. Support safe retries	Use request identifiers, remote references and reconciliation checks so a timeout does not create a duplicate document.
6. Test the full lifecycle	Create, submit, pay, reconcile, reverse and report. Include unavailable service, changed permissions and partial failure scenarios.

> Prefer one authoritative ledger. Updating two financial databases independently and hoping they stay synchronized creates reconciliation risk. A “single database only” requirement is a strong reason to choose the native route instead.

The adapter, sign-in and retry design above are recommendations, not out-of-the-box capabilities guaranteed by the REST API.

Sources: [2] Frappe REST API

Scope, dependencies and go-live decisions

Release in phases

Phase	Scope	Release condition
1. Foundation	Steps 1-5: scope, access, schema, ledger, opening data	Posting invariants and company isolation pass.
2. Working accounting	Steps 6-10: documents, settlement, controls and reports	End-to-end finance examples reconcile.
3. Pilot and launch	Steps 11-12: tests, migration and monitored rollout	Finance and operational owners accept the pilot.
4. Broader parity	FX, budgets/dimensions, stock, assets, payroll, localization and cheque workflows	Each extension has its own acceptance scenarios.

Decisions needed before development

- Choose native Judibas or separate ERPNext integration.
- Nominate the finance owner and agree release-one currency and business scope.
- Approve document and payment approval/correction permissions.
- Identify the opening-data source and migration cutoff.
- Prioritize later modules and define their accounting handover points.

Schedule and cost

This is a substantial accounting project, not a short UI port. Estimate the delivery schedule only after the scope, team, accountant availability and integration route are agreed. Track progress by demonstrated financial acceptance, not by screen count.

Sources and document boundary

Official sources checked on 21 October 2026. The proposed Laravel/PostgreSQL design and step diagrams are original recommendations for Judibas; they are not official Frappe migration instructions.

1. **Frappe architecture**
   https://docs.frappe.io/framework/user/en/basics/architecture
2. **Frappe REST API**
   https://docs.frappe.io/framework/user/en/api/rest
3. **ERPNext license**
   https://raw.githubusercontent.com/frappe/erpnext/develop/license.txt
4. **Accounting entries**
   https://docs.frappe.io/erpnext/accounting-entries
5. **Immutable ledger**
   https://docs.frappe.io/erpnext/immutable-ledger-in-erpnext
6. **Chart of accounts**
   https://docs.frappe.io/erpnext/chart-of-accounts
7. **Payment entry**
   https://docs.frappe.io/erpnext/payment-entry
8. **Accounting reports**
   https://docs.frappe.io/erpnext/accounting-reports

Related local reference

Frappe-Accounting-Guide.pdf describes the product features. This implementation roadmap explains how to organize a Judibas project based on those requirements.

> No production database, product access, financial table or deployment was changed while preparing this document. Code snippets, table concepts and workflows are planning material.